

Problem Set 5 - Repetition flow

Hello student,

the goal of this problem-set is try out the different approaches we have learned about repetition.

In particular, we want to practice the usage of `for` and `while` loops.

Assignment 5.1 - Armstrong numbers (optional)

Write a program that is able to find the first `N` Armstrong numbers, asking to the user the number `N`.

An Armstrong number is a number whose sum of the digits, each raised to the power of the number of digits, gives the original number as a result. The number 153 (3 digits) is for example an Armstrong number since:

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

The program must start checking numbers from 0 and stop when it has the `N` numbers as result. It must print all the armstrong number found and the maximum number checked.

Here an example of how the output **must** look like (in red the value inserted by the user):

```
Enter the number of Armstrong numbers to search for: 20

Armstrong number found (1): --> 0
Armstrong number found (2): --> 1
Armstrong number found (3): --> 2
Armstrong number found (4): --> 3
Armstrong number found (5): --> 4
Armstrong number found (6): --> 5
Armstrong number found (7): --> 6
Armstrong number found (8): --> 7
Armstrong number found (9): --> 8
Armstrong number found (10): --> 9
Armstrong number found (11): --> 153
Armstrong number found (12): --> 370
Armstrong number found (13): --> 371
Armstrong number found (14): --> 407
Armstrong number found (15): --> 1634
Armstrong number found (16): --> 8208
Armstrong number found (17): --> 9474
Armstrong number found (18): --> 54748
Armstrong number found (19): --> 92727
Armstrong number found (20): --> 93084

Found 20 Armstrong numbers in the first 93085 numbers checked
```

Assignment 5.2 - Triangles (optional)

Write a program that prints 2 of the 4 triangles displayed below.

The first one should be chosen between (a) and (b) while the second one should be chosen between (c) and (d).

Develop a simple menu that asks the user to choose the first triangle (a or b), prints the result, and then does the same for the second triangle (c or d)

You must use *for* loop(s) to generate them.

For each triangle, all the asterisks(*) must be printed using cycles.

(a)

```
*
**
***
****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

(b)

```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

(c)

```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

(d)

```

*
**
***
****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

Assignment 5.3 - Pi value (optional)

Print a table that shows the number of iteration and the approximated value of π for the first 10000 iterations.

The output must look exactly like this:

Iteration #	Value of Pi
-----	-----
0	4.0
1	2.666666666666667
2	3.466666666666667
3	2.8952380952380956
4	3.3396825396825403
...	

Hint: The formula to approximate π is: $\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$

Assignment 5.4 - Product odd numbers (optional)

Write a program that asks the user for two integer numbers. Then it must calculate the product between all the odd numbers included in the range defined by them. The boundaries are included. The request should be repeated if the second number is lower than the first one.

The output should look like this:

```
Lower bound:-5
Upper bound:-15

Upper bound not valid. Retry: 22

Product of all the odd numbers between -5 and 22: -206239658625
```

Attention: "min" and "max" are reserved keywords, use "_min" and "_max"

Assignment 5.5 - Minimum and Maximum (optional)

Write a program that asks the user for four non-integer numbers. Then the program must print the minimum and the maximum between the numbers.

The output should look like this:

```
Insert 4 non-integer numbers:
1.25
2.75
6.33
-4.25
Minimum: -4.25
Maximum: 6.33
```